CodInClub

Powered By BridgeLabz

Best Programming Practices in C#

General Guidelines

1. **Use static** for shared values and utility methods to reduce memory usage and avoid redundancy.
2. **Leverage this** to avoid ambiguity when initializing attributes.
3. **Declare readonly variables** for identifiers or constants that should remain unchanged.
4. **Use is operator** for safe type-checking and to prevent runtime errors during typecasting.

Sample Program 1: Bank Account System

Create a BankAccount class with the following features:

- **static**:
    - A static variable bankName shared across all accounts.
    - A static method GetTotalAccounts() to display the total number of accounts.
- **this**:
    - Use this to resolve ambiguity in the constructor when initializing AccountHolderName and AccountNumber.
- **readonly**:
    - Use a readonly variable AccountNumber to ensure it cannot be changed once assigned.
- **is operator**:
    - Check if an account object is an instance of the BankAccount class before displaying its details.

Sample Program 2: Library Management System

Create a Book class to manage library books with the following features:

- **static**:
    - A static variable LibraryName shared across all books.

  - A static method DisplayLibraryName() to print the library name.
- **this**:
  - Use this to initialize Title, Author, and ISBN in the constructor.
- **readonly**:
  - Use a readonly variable ISBN to ensure the unique identifier of a book cannot be changed.
- **is operator**:
  - Verify if an object is an instance of the Book class before displaying its details.

Sample Program 3: Employee Management System

Design an Employee class with the following features:

- **static**:
  - A static variable CompanyName shared by all employees.
  - A static method DisplayTotalEmployees() to show the total number of employees.
- **this**:
  - Use this to initialize Name, Id, and Designation in the constructor.
- **readonly**:
  - Use a readonly variable Id for the employee ID, which cannot be modified after assignment.
- **is operator**:
  - Check if a given object is an instance of the Employee class before printing the employee details.

Sample Program 4: Shopping Cart System

Create a Product class to manage shopping cart items with the following features:

- **static**:
  - A static variable Discount shared by all products.

  - A static method UpdateDiscount() to modify the discount percentage.

- **this**:
  - Use this to initialize ProductName, Price, and Quantity in the constructor.
- **readonly**:
  - Use a readonly variable ProductID to ensure each product has a unique identifier that cannot be changed.
- **is operator**:
  - Validate whether an object is an instance of the Product class before processing its details.

Sample Program 5: University Student Management

Create a Student class to manage student data with the following features:

- **static**:
  - A static variable UniversityName shared across all students.
  - A static method DisplayTotalStudents() to show the number of students enrolled.
- **this**:
  - Use this in the constructor to initialize Name, RollNumber, and Grade.
- **readonly**:
  - Use a readonly variable RollNumber for each student that cannot be changed.
- **is operator**:
  - Check if a given object is an instance of the Student class before performing operations like displaying or updating grades.

Sample Program 6: Vehicle Registration System

Create a Vehicle class with the following features:

- **static**:
  - A static variable RegistrationFee common for all vehicles.

  - A static method UpdateRegistrationFee() to modify the fee.
- **this**:
  - Use this to initialize OwnerName, VehicleType, and RegistrationNumber in the constructor.
- **readonly**:
  - Use a readonly variable RegistrationNumber to uniquely identify each vehicle.
- **is operator**:
  - Check if an object belongs to the Vehicle class before displaying its registration details.

Sample Program 7: Hospital Management System

Create a Patient class with the following features:

- **static**:
  - A static variable HospitalName shared among all patients.
  - A static method GetTotalPatients() to count the total patients admitted.
- **this**:
  - Use this to initialize Name, Age, and Ailment in the constructor.
- **readonly**:
  - Use a readonly variable PatientID to uniquely identify each patient.
- **is operator**:
  - Check if an object is an instance of the Patient class before displaying its details.
